

Reviewer Pack — Schatten-1/Schatten-4 Ratio Lower-Bounds AND-Function Commun...

Entry 87f5d4a9eef6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 17:24:03 UTC

Schatten-1/Schatten-4 Ratio Lower-Bounds AND-Function Communication

Entry ID: 87f5d4a9eef6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 17:24:03 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative L^p geometry — Schatten- p norm ratios in the Pisier / Lust-Piquard framework, specifically the (S_1, S_4) stable-rank invariant $\tau(M) = \log_2(\|M\|_{S_1}^2 / \|M\|_{S_4}^2)$. This corner of operator-space theory is rarely applied to communication complexity beyond Forster’s operator-norm sign-rank bound; an arXiv search for ‘Schatten 4 norm’ AND ‘communication complexity’ returns <5 hits, all on unrelated invariants (e.g., spectral norm or γ_2). **Field B** (complexity object): Randomized two-party communication complexity of AND-functions $M_g(x \wedge y)$ on $\{0,1\}^n \times \{0,1\}^n$, with the prototype $g = \text{AND-of-NOTs}$ giving $M = M_{\text{DISJ}_n}$ (Razborov 1992 $\Omega(n)$ regime). The conjecture instantiates the $\text{COMM}_{\text{DISJ}}$ tensor-invariant program and parallels the spectral-expansion mechanism underlying Tseitin-on-expander resolution width lower bounds (Ben-Sasson-Wigderson, Atserias-Bonet-Esteban).

Statement:

For $g: \{0,1\}^n \rightarrow \{0,1\}$, let $M_g \in \{0,1\}^{2^n \times 2^n}$ be the AND-function communication matrix $M_g[x,y] = g(x \wedge y)$. With σ_i = singular values of M_g , define $\tau(M_g) := \log_2((\sum_i \sigma_i)^2 / (\sum_i \sigma_i^4)^{1/2}) = 2 \cdot \log_2(\|M_g\|_{S_1}) - \log_2(\|M_g\|_{S_4}^2)$. Conjecture (A): for every AND-function, $\text{CC}_{\mathbb{R}}^{\{1/3\}}(M_g) \geq (1/8) \tau(M_g) - 2$; (B): for M_{DISJ_n} ($g(z) = \prod_i (1 - z_i)$), $\tau(M_{\text{DISJ}_n}) = n \cdot \log_2(5/\sqrt{7}) \approx 0.918n$, exactly. A single AND-function M_g with $\tau(M_g) > 8 \cdot D^{\text{cc}}(M_g) + 16$, or any n with $\tau(M_{\text{DISJ}_n}) < 0.7n$, falsifies.

Rationale (proposer’s reasoning):

The DISJ matrix factors as a tensor power of the 2×2 block $(1,1;1,0)$ with golden-ratio spectrum $\{\varphi, -1/\varphi\}$, so its Schatten norms have closed forms ($\|\cdot\|_{S_p}^p = (\varphi^p + \varphi^{-p})^n$), giving $\tau = \Theta(n)$ by exact computation rather than spectral inequalities. The (S_1, S_4) ratio strengthens Forster’s (S_2, S_∞) ratio: it is more sensitive to mid-spectrum mass while remaining sub-multiplicative under tensor product, the same algebraic property that makes Tseitin-on-expanders amenable to width-via-expansion arguments. If proved, conjecture (A) yields a clean, calculable tensor invariant that matches Razborov’s $\Omega(n)$ bound on a wide AND-function class and would import noncommutative L^p techniques into proof-complexity-flavored CC lower bounds.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7dcaa2120433d488

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n=4..8$ with 30 seeds for random/tensor families: (a) for every n , $|\tau(M_DISJ_n) - n \cdot \log_2(5/\sqrt{7})| \leq 1e-6$ and $\tau(M_DISJ_n)/n \in [0.7, 1.0]$; (b) for every instance in families (ii)-(iv), $\tau(M_g) \leq 8 \cdot \text{ub}(M_g) + 16$. Support requires 100% of instances pass; one violation falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL PROOFS	SAFE	0.85	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Schatten norm communication complexity AND function disjointness - Schatten-4 norm singular values lower bound communication matrix - stable rank Schatten ratio sign-rank Forster disjointness

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import sys
import random
import math
import json
from fractions import Fraction

def matrix_mult(a, b):
    n = len(a)
    m = len(b[0])
    p = len(b)
    result = [[0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
            for k in range(p):
                result[i][j] += a[i][k] * b[k][j]
    return result
```

```

def matrix_transpose(a):
    return [list(row) for row in zip(*a)]

def matrix_norm(a):
    return math.sqrt(sum(sum(x**2 for x in row) for row in a))

def svd(a):
    a = [[float(x) for x in row] for row in a]
    m = len(a)
    n = len(a[0]) if m > 0 else 0
    u = [[0.0 for _ in range(m)] for _ in range(m)]
    vt = [[0.0 for _ in range(n)] for _ in range(n)]
    s = [0.0] * min(m, n)

    # Simple SVD implementation for small matrices
    at = matrix_transpose(a)
    ata = matrix_mult(at, a)
    u = [[0.0 for _ in range(m)] for _ in range(m)]
    for i in range(m):
        u[i][i] = 1.0

    # Power iteration for dominant singular value
    b = [row[:] for row in a]
    for i in range(min(m, n)):
        if i >= len(b) or i >= len(b[0]):
            break
        s[i] = matrix_norm([row[i] for row in b])
        if s[i] == 0:
            continue
        for j in range(len(b)):
            if j >= len(b[0]):
                break
            b[j][i] /= s[i]
        if i + 1 < min(m, n):
            q = [row[i+1] for row in b]
            for j in range(i+1):
                q[j] = 0.0
            q_norm = matrix_norm(q)
            if q_norm > 0:
                for j in range(len(q)):
                    q[j] /= q_norm
                for j in range(len(b)):
                    if j >= len(b[0]):
                        break
                    b[j][i+1] -= q[j] * q_norm

    # Construct vt
    for i in range(min(m, n)):
        if i >= len(vt) or i >= len(vt[0]):
            break
        vt[i][i] = 1.0

    return u, s, vt

def compute_tau(matrix):
    u, s, vt = svd(matrix)

```

```

s1 = sum(s)
s4 = sum(x**4 for x in s) ** (1/4)
if s4 == 0:
    return 0.0
tau = math.log2(s1**2 / s4)
return tau

def generate_disj_matrix(n):
    size = 2 ** n
    matrix = [[0] * size for _ in range(size)]
    for x in range(size):
        for y in range(size):
            z = x & y
            matrix[x][y] = 1 if z == 0 else 0
    return matrix

def generate_and_matrix(n, seed):
    random.seed(seed)
    size = 2 ** n
    matrix = [[0] * size for _ in range(size)]
    for x in range(size):
        for y in range(size):
            z = x & y
            matrix[x][y] = 1 if z == (1 << n) - 1 else 0
    return matrix

def generate_random_and_matrix(n, seed):
    random.seed(seed)
    size = 2 ** n
    matrix = [[0] * size for _ in range(size)]
    for x in range(size):
        for y in range(size):
            z = x & y
            matrix[x][y] = random.choice([0, 1])
    return matrix

def generate_tensor_and_matrix(n, seed):
    random.seed(seed)
    size = 2 ** n
    matrix = [[0] * size for _ in range(size)]
    for x in range(size):
        for y in range(size):
            z = x & y
            matrix[x][y] = (z % 3) + 1
    return matrix

def compute_cc_upper_bound(matrix):
    n = int(math.log2(len(matrix)))
    rank = sum(1 for s in svd(matrix)[1] if s > 1e-10)
    return min(2 * n, math.ceil(math.log2(rank + 1)) + 4 * math.log2(len(matrix)))

def run_trial(seed):
    random.seed(seed)
    n = random.choice([4, 5, 6, 7, 8])
    instances_tested = 0
    conjecture_holds = True

```

```

counterexample = ""

# Test DISJ matrix
matrix = generate_disj_matrix(n)
tau = compute_tau(matrix)
expected_tau = n * math.log2(5 / math.sqrt(7))
if abs(tau - expected_tau) > 1e-6 or tau / n < 0.7 or tau / n > 1.0:
    conjecture_holds = False
    counterexample = f"DISJ matrix tau={tau} not in [0.7n,1.0n] for n={n}"

# Test AND matrices
for family in [generate_and_matrix, generate_random_and_matrix, generate_tensor_and_matrix]:
    matrix = family(n, seed)
    tau = compute_tau(matrix)
    ub = compute_cc_upper_bound(matrix)
    if tau > 8 * ub + 16:
        conjecture_holds = False
        counterexample = f"{family.__name__} tau={tau} > 8*ub+16 for n={n}"
        break

instances_tested += 1

return {
    "metric_name": "tau",
    "metric_value": tau,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    results = []
    metric_values = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps({'seed': seed, **result})}")
        results.append(result)
        metric_values.append(result["metric_value"])

    mean_metric = sum(metric_values) / len(metric_values) if metric_values else 0
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results) if results

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[first_failing_seed]['counterexample']}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_624caab1.py", line 177, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_624caab1.py", line 145, in run_trial
    tau = compute_tau(matrix)
           ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_624caab1.py", line 84, in compute_tau
    u, s, vt = svd(matrix)
                ^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_624caab1.py", line 55, in svd
    s[i] = matrix_norm([row[i] for row in b])
                ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_624caab1.py", line 33, in matrix_norm
    return math.sqrt(sum(sum(x**2 for x in row) for row in a))
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_624caab1.py", line 33, in <genexpr>
    return math.sqrt(sum(sum(x**2 for x in row) for row in a))
                          ^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: 'float' object is not iterable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in the custom SVD/matrix_norm implementation before any data was produced, so neither the support nor falsification criteria can be evaluated.
| next: Fix the matrix_norm/SVD helper (use `numpy.linalg.svd` on the 2D matrix instead of the buggy nested-iteration norm) and rerun the n=4..8 sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	507456
2	preregistration	claude_max	opus	0	0	7765
3	novelty	claude_max	opus	0	0	4406
4	novelty	claude_max	opus	0	0	9003
5	test_gen	mistral	codestral-latest	0	0	319115
6	test_gen	mistral	codestral-latest	0	0	312864
7	test_gen	mistral	codestral-latest	0	0	310827
8	test_gen	mistral	codestral-latest	0	0	312176
9	judge	claude_max	opus	0	0	4295

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
---	-------	----------	-------	-----------	-----	--------------

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1787907 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in *research/audit/87f5d4a9eef6.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/87f5d4a9eef6.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/87f5d4a9eef6.tar.gz* (if generated)